Report - Assignment 10

Name: Hima Sai Chandana | Roll No: 2401MC37

1. Memory Management: Physical Memory Tracking (Q1)

- **Implementation:** Modified kalloc.c by adding counters for total allocated (total_allocated_pages) and freed pages (total_freed_pages). Incremented these inside kalloc() and kfree() respectively under kmem.lock. Implemented sys_memstats() to expose these metrics to user space.
- **Observations:** Heap allocations via sbrk() dynamically update active allocation counts. Releasing memory immediately decrements active counts and restores system availability without memory leaks.

2. Virtual Memory: Virtual-to-Physical Address Translation (Q2)

- **Implementation:** Implemented vtop(int virtual_address) system call. It uses walkpgdir() to traverse the page table hierarchy. If PTE_P (Present bit) is set, it extracts the physical frame address (PTE_ADDR(*pte)) and combines it with the lower 12-bit page offset (virtual_address & 0xFFF). Returns -1 if unmapped or invalid.
- **Observations:** Page translation maintains consistent intra-page offsets while providing isolation between process virtual address spaces. Unmapped accesses are safely caught by checking PTE_P.

3. Paging: Page Table Management & Page Protection (Q3)

- **Implementation:** Created pageinfo() to query page mapping and return flag states (PTE_P, PTE_W, PTE_U). Implemented setreadonly() to clear the write flag (*pte &= ~PTE_W) and invoked lcr3() to flush the CPU TLB.
- **Observations:** Attempting to write to a page after clearing PTE_W immediately triggers hardware Trap 14 (Page Fault), verifying hardware-level memory protection.

4. Page Replacement Algorithms (Q4)

- **Implementation:** Implemented pagereplace.c evaluating **FIFO** (replaces oldest page in queue), **LRU** (replaces page unaccessed for longest time), and **Optimal** (replaces page used furthest in future) across a 20-reference string: 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1.

Performance Comparison Table

Frames	Algorithm	Hits	Faults	Hit Ratio (%)	Fault Ratio (%)
3	FIFO / LRU / OPT	5 / 8 / 11	15 / 12 / 9	25% / 40% / 55%	75% / 60% / 45%
4	FIFO / LRU / OPT	10 / 12 / 14	10 / 8 / 6	50% / 60% / 70%	50% / 40% / 30%
5	FIFO / LRU / OPT	13 / 13 / 15	7 / 7 / 5	65% / 65% / 75%	35% / 35% / 25%

Key Observations

1. **Optimal (OPT)** consistently yields the lowest page fault count across all frame sizes, providing the benchmark lower bound.
2. **LRU** outperforms **FIFO** on 3 and 4 frames by leveraging temporal locality, but both converge at 5 frames for this reference string.
3. Increasing available frame capacity directly decreases page fault frequency across all three algorithms.